

OrderPulse — Step 5: Observability & Log Aggregation

Technical Instruction & Execution Blueprint for AI Developer Agents

OBJECTIVE

Configure centralized log aggregation using Grafana and Loki. Ensure structured uber-go/zap logs from Inventory, Order, and Worker services are indexed for end-to-end trace correlation.

Step 5 Tasks

- **1. Zap Log Exporter:** Configure microservices to export structured JSON logs to Loki (via HTTP push or Docker logging driver).
- **2. Context Enrichment:** Ensure all logs contain standard context fields: `service_name`, `trace_id`, and `order_id`.
- **3. Datasource Configuration:** Add Loki (`http://loki:3100`) as an active datasource in Grafana UI.
- **4. Grafana Dashboard:** Build a custom dashboard with LogQL queries allowing log filtering by `order_id` across all microservices.

LogQL Query Reference

```
# Query all microservice logs
{container_name=~"orderpulse-.*"} | json

# Trace a single order across all services
{container_name=~"orderpulse-.*"} | json | order_id = "ord-12345"

# Filter error logs across the system
{container_name=~"orderpulse-.*"} | json | level = "error"
```

Verification Commands

```
# Check Loki status
curl -s http://localhost:3100/ready

# Check indexed labels
curl -s http://localhost:3100/loki/api/v1/label/service_name/values

# Trigger sample order request
grpcurl -plaintext -d '{"user_id": "usr-1", "product_id": "prod-101", "quantity": 1}' localhost:5005
```